LAB -4 Complexity Comparison

Name: Priyanshu Mandal **Roll No.:** 2401201035

Specialisation: BCA — Artificial Intelligence and Data Science **Section:** A

Lab: Lab 4 — Smart Optimization Assistant using Dynamic Programming

Execution time, space usage and number of subproblems / DP states

Theoretical comparison

Algorithm	Time complexity	Space complexity	DP subproblems / states
Longest Common Subsequence	O(m × n)	O(m × n)	(m+1)(n+1) table cells; m×n transitions
0/1 Knapsack	O(n × W)	O(n × W)	(n+1)(W+1) table cells; n×(W+1) transitions
Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$	$O(n^2)$ interval states; $O(n^3)$ split evaluations

Code –

```
from time import perf_counter import
tracemalloc


def lcs(x: str, y: str):     m, n = len(x),
len(y)     dp = [[0] * (n + 1) for _ in
range(m + 1)]     for i in range(1, m + 1):
for j in range(1, n + 1):             if x[i - 1]
== y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
i, j, seq = m, n, []     while i > 0 and j > 0:
if x[i - 1] == y[j - 1]:
            seq.append(x[i - 1]); i -= 1; j -= 1
elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
else:
j -= 1
    return dp[m][n], ''.join(reversed(seq)), dp


def knapsack_01(weights, values, capacity):
    if len(weights) != len(values):
        raise ValueError("weights and values must have equal length")
if capacity < 0 or any(w < 0 for w in weights):
```

```
        raise ValueError("weights and capacity must be non-negative")
n = len(weights)     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
        for cap in range(capacity + 1):             if weights[i - 1] <= cap:
dp[i][cap] = max(values[i - 1] + dp[i - 1][cap - weights[i - 1]],
dp[i - 1][cap])             else:
                dp[i][cap] = dp[i - 1][cap]
chosen, cap = [], capacity     for i in
range(n, 0, -1):         if dp[i][cap] !=
dp[i - 1][cap]:
chosen.append(i - 1)             cap -=
weights[i - 1]     chosen.reverse()
return dp[n][capacity], chosen, dp


def matrix_chain_order(dims):     if len(dims)
< 2 or any(d <= 0 for d in dims):
        raise ValueError("Provide at least two positive dimensions")
n = len(dims) - 1     cost = [[0] * (n + 1) for _ in range(n + 1)]
split = [[0] * (n + 1) for _ in range(n + 1)]     for length in range(2,
n + 1):         for i in range(1, n - length + 2):
            j = i + length - 1
            cost[i][j] = float("inf")
for k in range(i, j):
                q = cost[i][k] + cost[k + 1][j] + dims[i-1] * dims[k] * dims[j]
if q < cost[i][j]:
                    cost[i][j], split[i][j] = q, k
return cost[1][n], cost, split


def optimal_parenthesization(split, i, j):
    if i == j:
        return f"A{i}"
k = split[i][j]
    return f"({optimal_parenthesization(split, i, k)} × {optimal_parenthesization(split, k+1, j)})"


def measure(fn, *args):
    """Return result, wall-clock seconds, and peak traced Python memory in KiB."""
tracemalloc.start()     start = perf_counter()     result = fn(*args)     elapsed =
perf_counter() - start
```

```
    _, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()     return result, elapsed,
peak / 1024
```

Measured performance

The table below summarizes measured runtime and peak traced Python memory. These are experimental observations from the preparation environment; results vary across devices and Python versions.

0/1 Knapsack — largest tested input 60: 0.007788 seconds; peak traced memory 296.29 KiB; recorded DP table cells 14,701.

LCS — largest tested input 60: 0.000657 seconds; peak traced memory 31.42 KiB; recorded DP table cells 3,721.

Matrix Chain — largest tested input 30: 0.016635 seconds; peak traced memory 30.52 KiB; recorded DP table cells 961.

Notes

m and n are the two sequence lengths; W is knapsack capacity. Knapsack's O(nW) bound is pseudo-polynomial. LCS and Knapsack table sizes include the base row and column. Matrix Chain Multiplication stores $O(n^2)$ interval states and evaluates $O(n^3)$ candidate splits.